Software Architecture Description

afetbilgi.com

Alperen Eynallı, 2521573
Bahram Abdullayev, 2489086
Group 14

May 2023

Contents

List of Figures

List of Tables

1 Introduction

The document is the Software Architecture Description of the website afetbilgi.com, a website that is created by METU students and graduates to provide important information about the 6 February 2023 Pazarcık Earthquake.

1.1 Purpose of the System

After 6 February 2023 Pazarcık Earthquake, it is very difficult for people to find and use important information, such as links, several locations, numbers and so on, through the internet. So, the aim of the system is to provide verified information via understandable interface about the 6 February 2023 Pazarcık Earthquake. The basic principles of the website are accuracy, speed, and simplicity. Since the earthquake is an emergency situation, accuracy and verification of information are very important and afetbilgi.com is careful to use only verified information.

1.2 Scope

The scope of the website project can be listed as follows:

- The system provides important locations such as safehouses, gas stations, hospitals, and many more that are useful to victims of the earthquake.
- The system provides simple and easy to use interface to users.
- The system provides maps that can help people locate and find places easier.
- The system provides important information such as phone numbers, donation links, etc.

1.3 Stakeholders and their concerns

1. Users: Users' primary goal is to quickly access reliable information about the natural disaster. Users can access the website and view the pages created by developers. They can use every feature provided by the website for free.

2. Developers: Developers' main concerns are to access reliable data and build an accessible fast website for users and help the country. They are the owner and creators of the project and are responsible for everything. They create the pages and get the data from the validators.

3. Validators: Validators' aim and role is to provide verified reliable data to developers and help their country. They get the verified data from organizations, twitter and other social media platforms.

2 References

This document is prepared with respect to IEEE 29148 2018-11 standard: 29148 2018-11 - ISO/IEC/IEEE International Standart - Systems and software engineering - life cycle processes - requirements engineering.

References

[1] afetbilgi.com

3 Glossary

User	A person that visits the website
Developer	Manages and develops everything about the website.
Typescript	TypeScript is an open-source programming language developed and maintained by Microsoft. It is a strict syntactical superset of JavaScript, and adds optional static typing to the language. TypeScript is designed for development of large applications and transcompiles to JavaScript. As TypeScript is a superset of JavaScript, existing JavaScript programs are also valid TypeScript programs.
Python	Python is an interpreted, object-oriented, high-level programming language with dynamic semantics. Its high-level, built-in data structures, combined with dynamic typing and dynamic binding, make it very attractive for Rapid Application Development, as well as for use as a scripting language to connect existing components together.
Javascript	JavaScript is a high-level, interpreted programming language that conforms to the ECMAScript specification. JavaScript has curly-bracket syntax, dynamic typing, prototype-based object-orientation, and first-class functions.
Amazon Web Services	Amazon Web Services (AWS) is a subsidiary company of Amazon, offering pay-as-you-go APIs and on-demand cloud computing services. AWS is headquartered in Seattle, Washington, and was founded in 2006. The company offers its services in eighty-four availability zones, in twenty-six geographic regions around the world. AWS's cloud infrastructure is intended to be a scalable and low-cost platform and is used in 190 countries around the world.
GitHub	GitHub, Inc. is a provider of Internet hosting for software development and version control using Git. It offers the distributed version control and source code management functionality of Git, plus its own features.
React	React (also known as React.js or ReactJS) is a JavaScript library for building user interfaces. React can be used to create single-page software applications for web and mobile platforms. The open-source library is maintained by Meta (formerly Facebook) and a community of software engineers. Although React is a library rather than a language, it is widely used in web development.

Table 1: Definitions

4 Architectural Views

4.1 Context View

4.1.1 Stakeholders' uses of this view

There are two main stakeholders of the system: the users, and developers. The users use this view to understand how afetbilgi.com works

and how the data is provided to them. Developers use this view to determine how to improve and extend the existing system.

4.1.2 Context Diagram

AfetBilgi is not a part of large system, and it is a simple system. In this viewpoint, context of the system with all actors are defined in general and detailed viewpoints. In the context diagram, actors and their interaction with aftebilgi.com will be explained in general terms. Use case diagrams and the detailed explanations of some possible use case of the system will be specified below the context diagram.

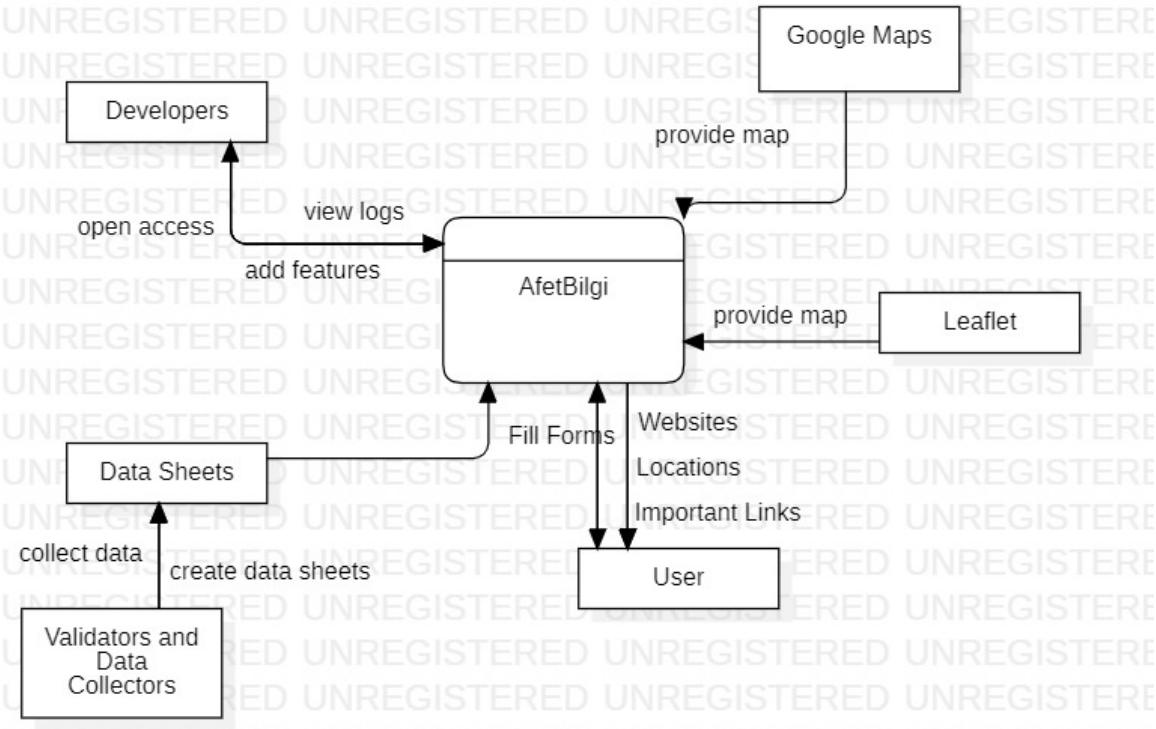

Figure 1: Context Diagram for Afetbilgi

- Developers view logs and can add features as the validators collect data after they accept the data as valid and reliable.
- Users can be provided with important links, sources, websites, locations, and related stuff. Also, they can fill out the forms on the website about a particular topic.
- Google Maps provide maps when users click to see some locations after some steps on the website.
- If the user clicks to see the map in the main interface of the system, Leaflet provides a map.
- Validators and data collectors create and provide data sheets to the developers after collecting and validating gathered data.

4.1.3 External Interfaces

External interfaces of the system includes User, Developer and Data Collector interfaces. Here is the class diagram for external interfaces:

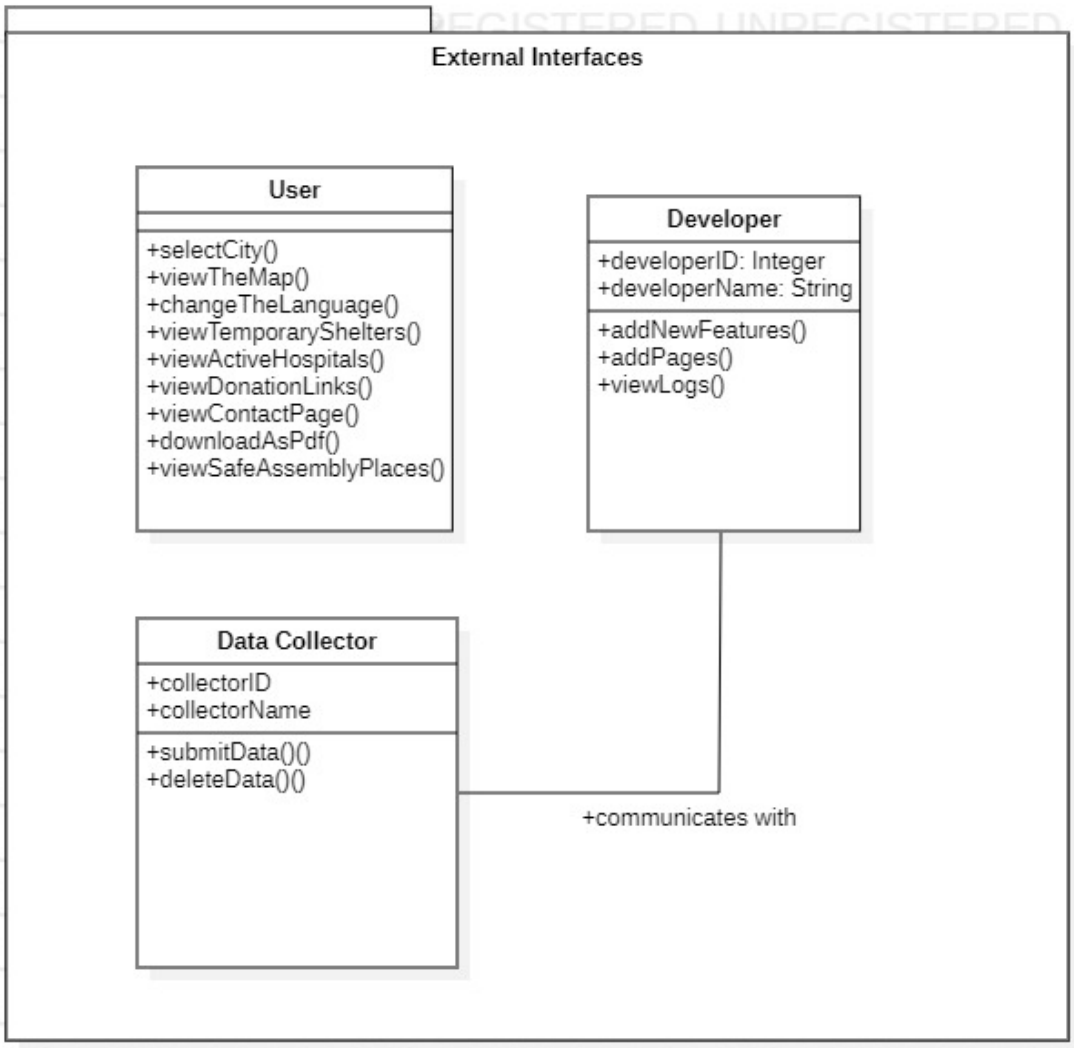

Figure 2: External Interfaces

selectCity()	Open a list of cities to select
viewTheMap()	Open the map provided by Leaflet API
changeTheLanguage()	Change the language of the website
viewTemporaryShelters()	Open the page containing list of temporary shelters
viewActiveHospitals()	Open the page containing list of active hospitals
viewDonationLinks()	Open the page containing list of donation links
viewContactPage()	Open the page containing the contact info
downloadAsPdf()	Open a list of cities to download info about that city as pdf file
viewSafeAssemblyPlaces()	Open a page contatining list of safe assembly places
addNewFeatures()	Add new features to the website
addPages()	Add new page to the website
viewLogs()	Open the logs file
submitData()	Submit the data sheet
deleteData()	Delete the data sheet

Table 2: External Interface Operation Descriptions

4.1.4 Interaction scenarios

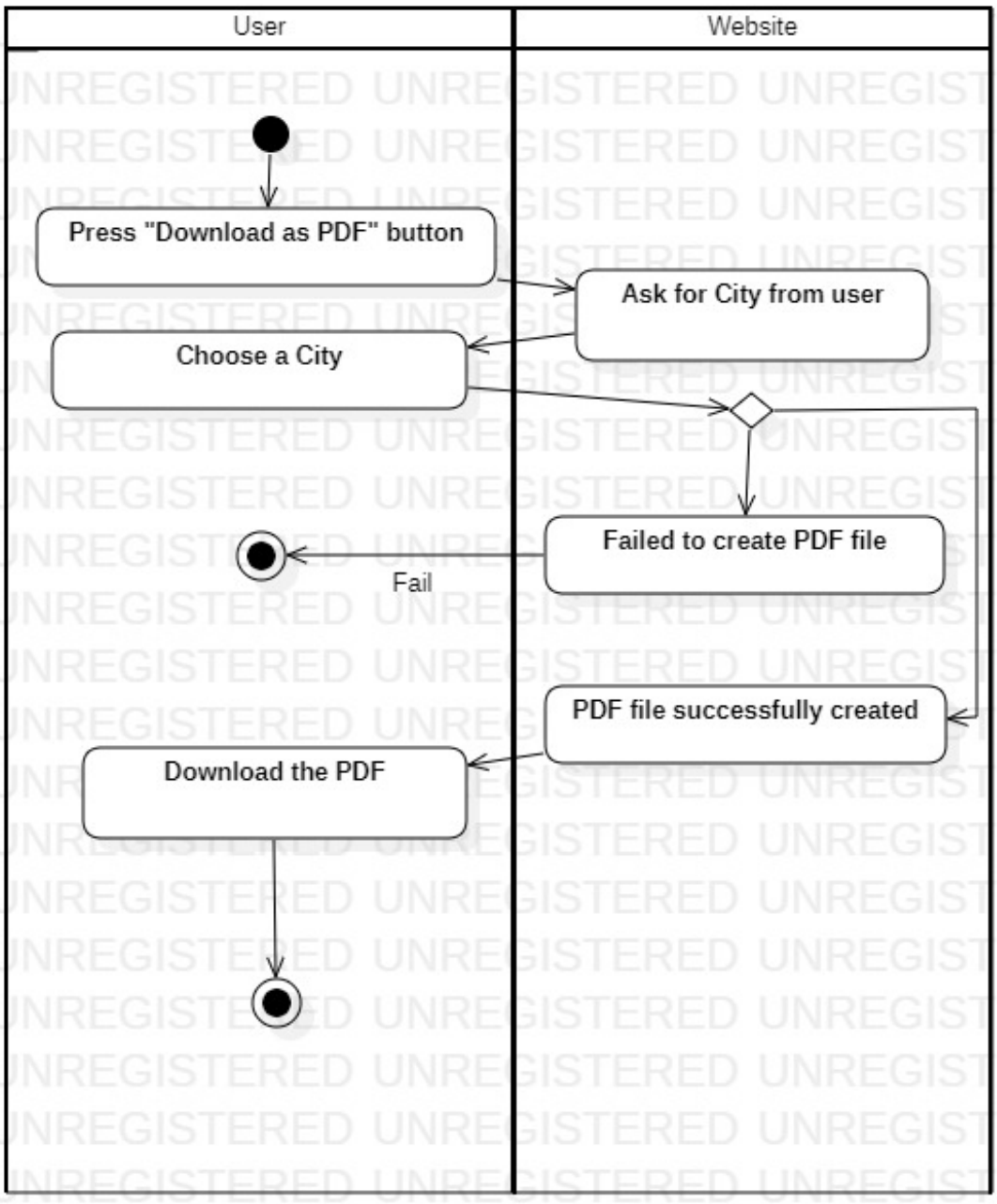

Figure 3: Activity Diagram 1

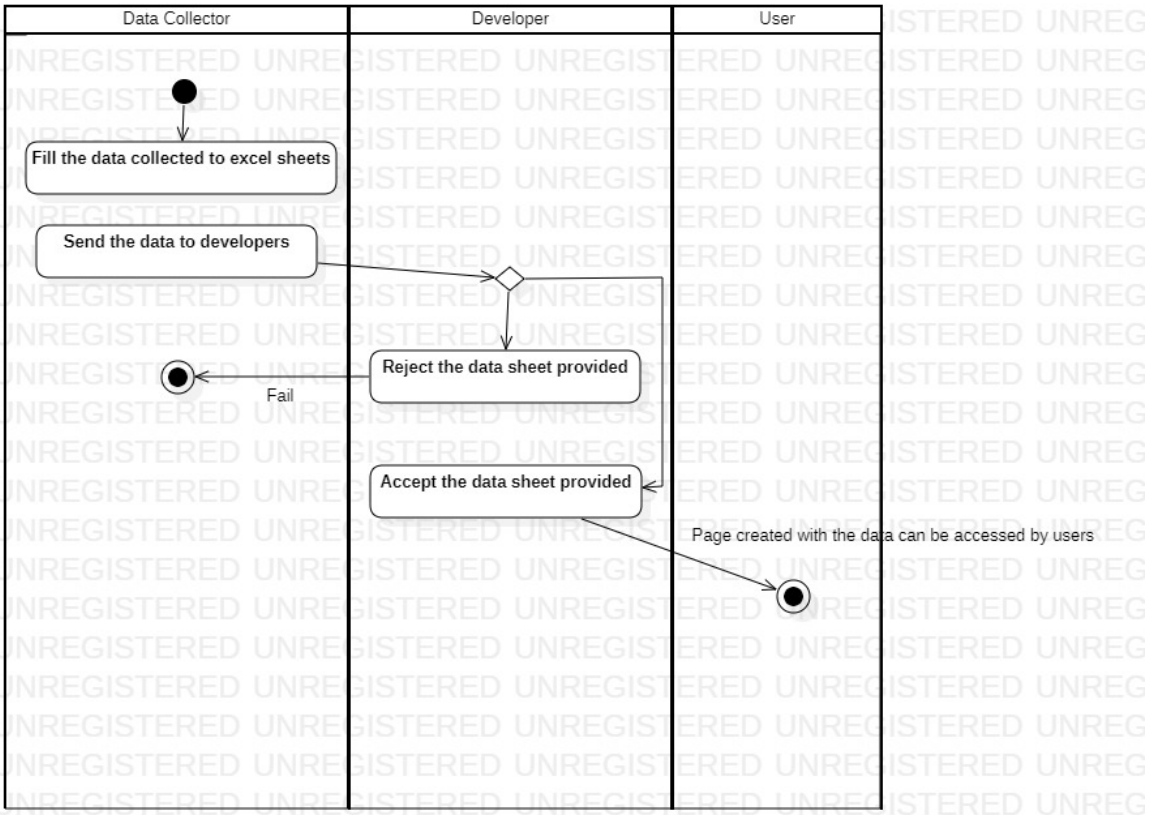

Figure 4: Activity Diagram 2

4.2 Functional View

4.2.1 Stakeholders' uses of this view

In this viewpoint, different components of the system, internal interfaces within the system, their responsibilities and interactions will be described. Functional view is very significant to the stakeholders, since it provides an idea of the functionalities of the system and forms the shape of other viewpoints.

4.2.2 Component Diagram

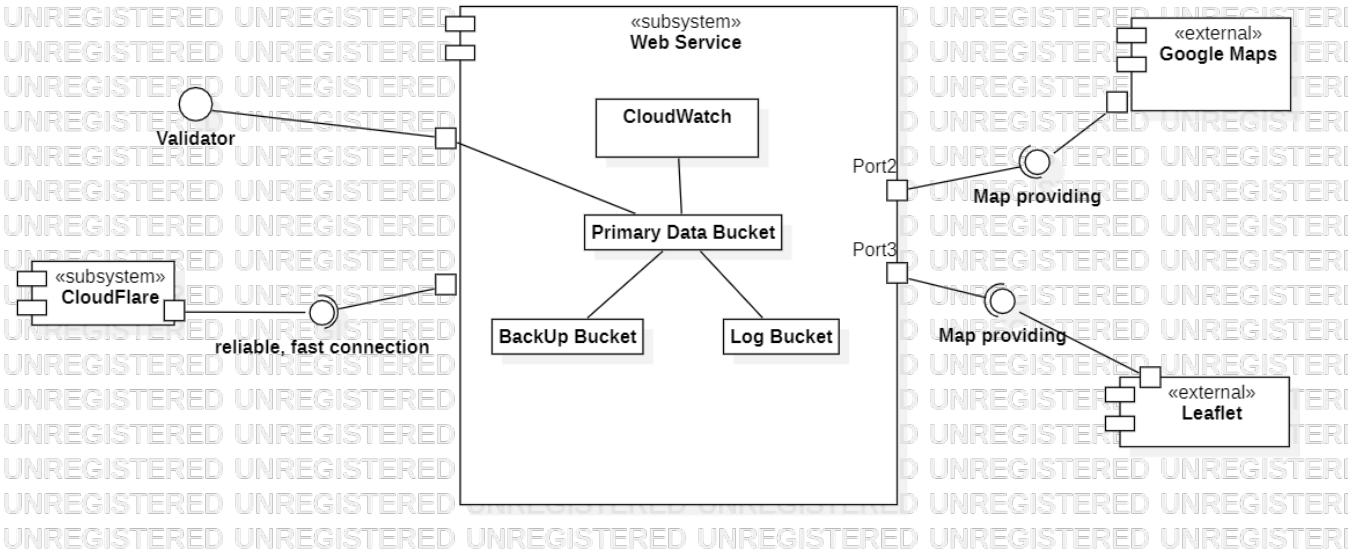

Figure 5: Component Diagram

- Our system contains several subsystems, such as Web Services and CloudFlare, which involve the main work of the system.
- There are some external interfaces, such as Google Map, Leaflet, Validators, and Data Collectors.
- Web Service as a subsystem includes parts, including CloudWatch, Primary Data Bucket, BackUp Bucket, and Log Bucket, which are all inter-related.
- Web Service correlate with Cloudflare to provide secure, reliable, and fast connections to users.
- Validators correlates with subsystem Web Service and their role is to provide verified reliable data to developers and help their

country. They get the verified data from organizations, twitter and other social media platforms.

- Google Maps and Leaflet as external interfaces provide maps to users in seperate parts of the system.

4.2.3 Internal Interfaces

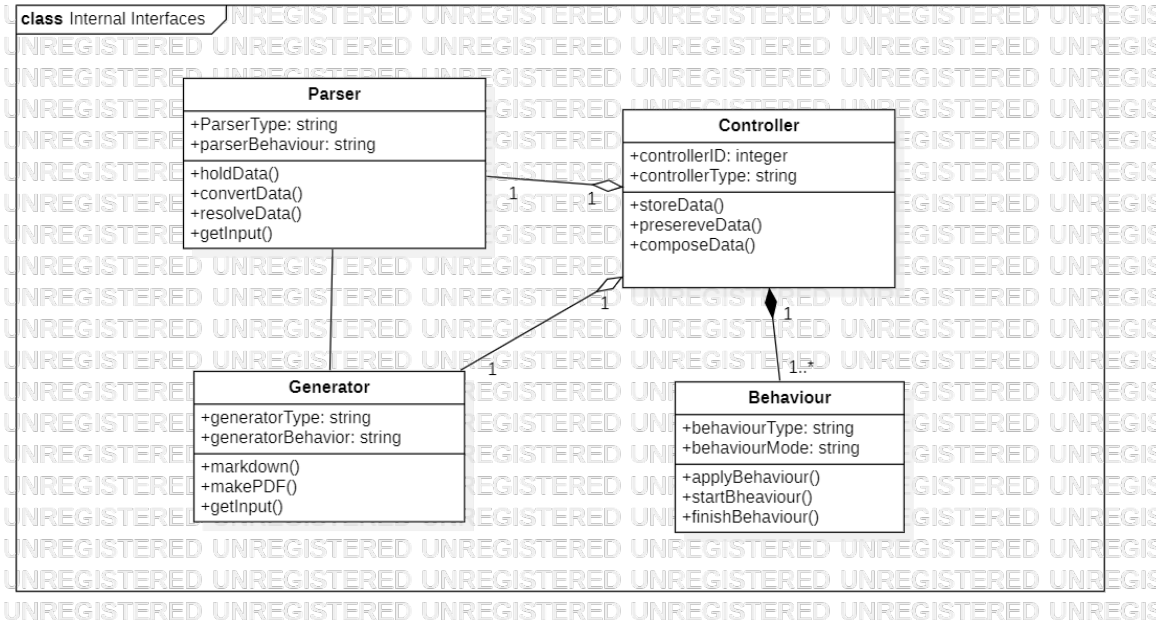

Figure 6: Internal Interfaces Class Diagram

- Parser is a program that takes input data (usually source code) and converts it into a structured format. The purpose of parsing in Python is to transform code into a machine-readable format, allowing for further processing and analysis.

- Generator is a program that take input data and convert the data to particular form in order to make PDF file format.

- Behavior is characterized as the behaviour of the user while using the system.
- Controller will be responsible for composing the corresponding parts of the system. It will be responsible for storing the needed data in the system and preserving the integrity of the system.

4.2.4 Interaction Patterns

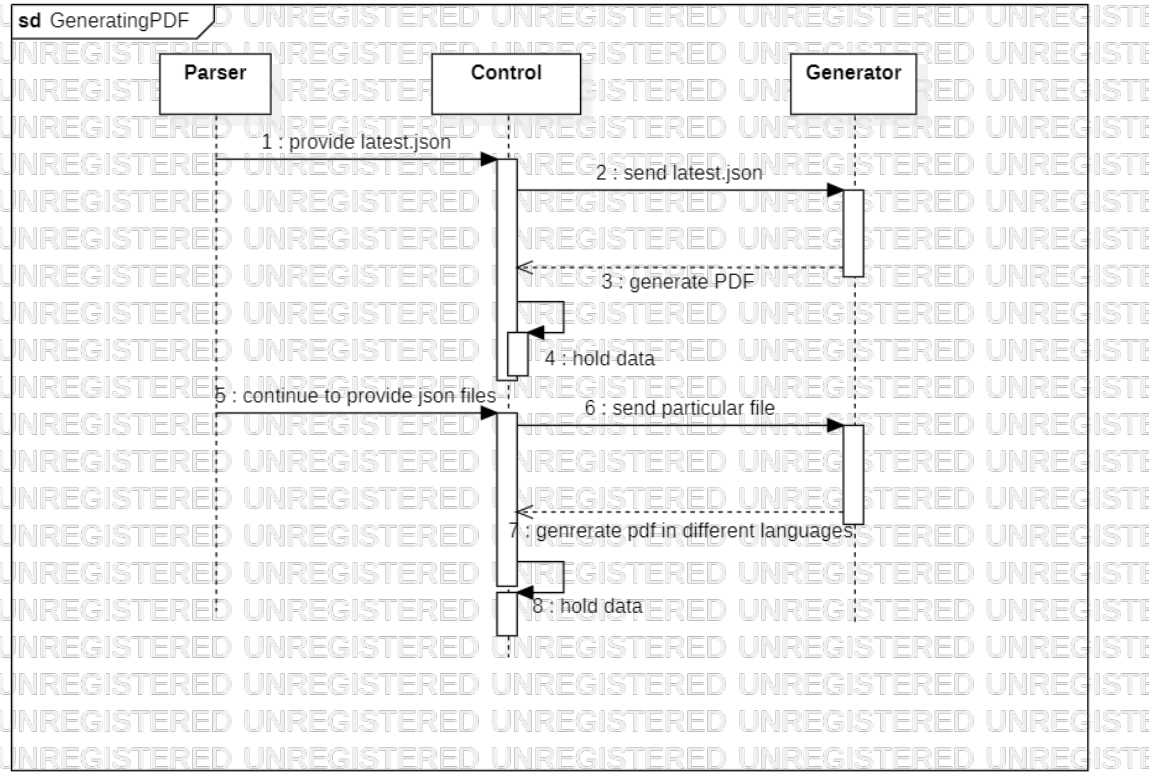

Figure 7: Sequence Diagram 1

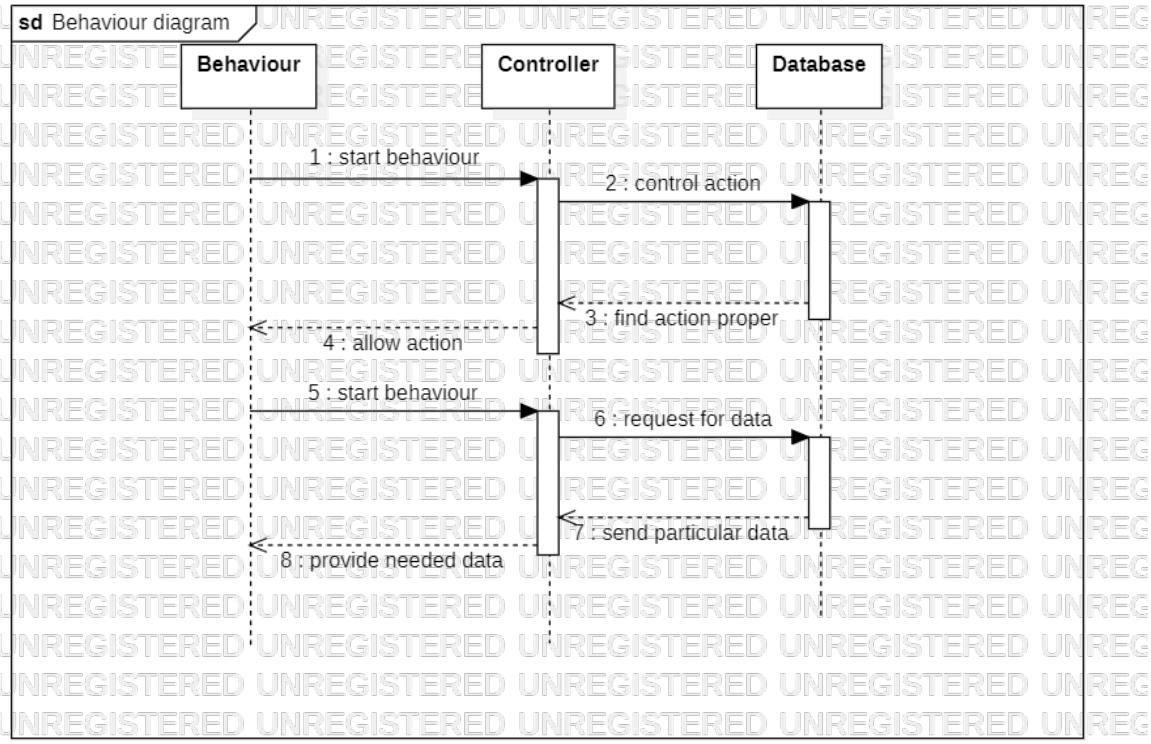

Figure 8: Sequence Diagram 2

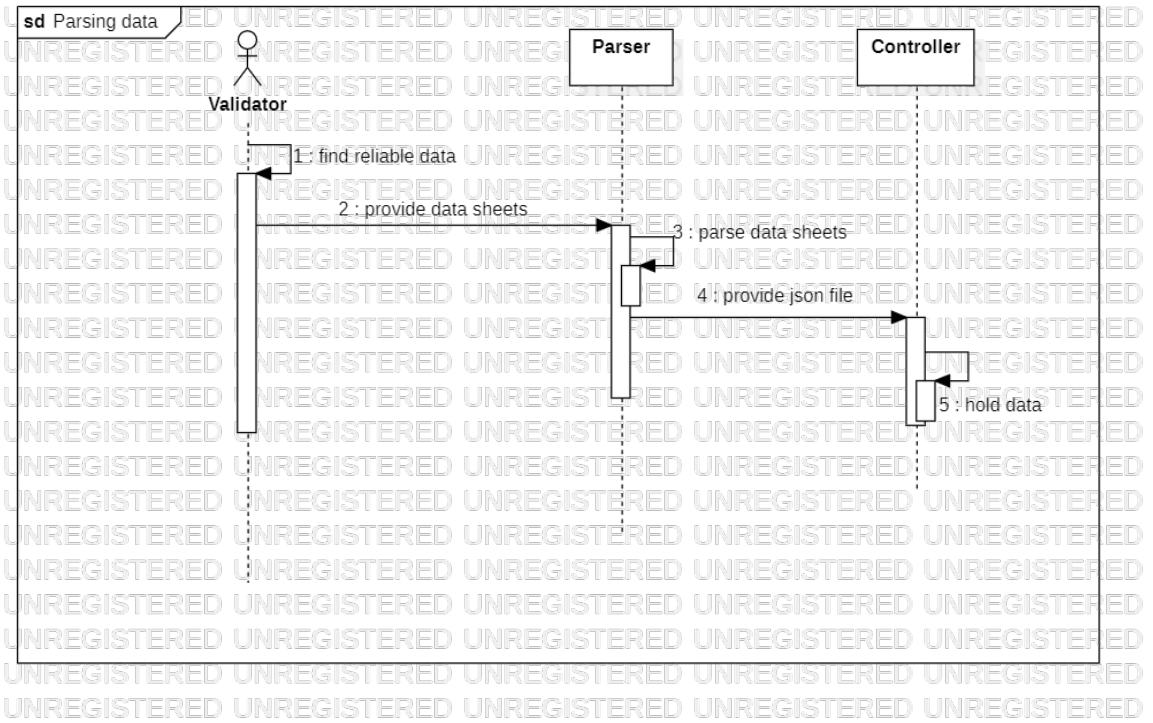

Figure 9: Sequence Diagram 3

4.3 Information View

4.3.1 Stakeholders' uses of this view

There are two main stakeholders of the system: Users and developers. Users use this view to understand how the data provided to them is related to each other and how it's formed. Users can understand what is stored in the system and its limitations. Developers use this view to see the relations of the db and its limitations. Also, they use this view to plan how to extend the db and the system without causing anomalies on the db.

4.3.2 Database Class Diagram

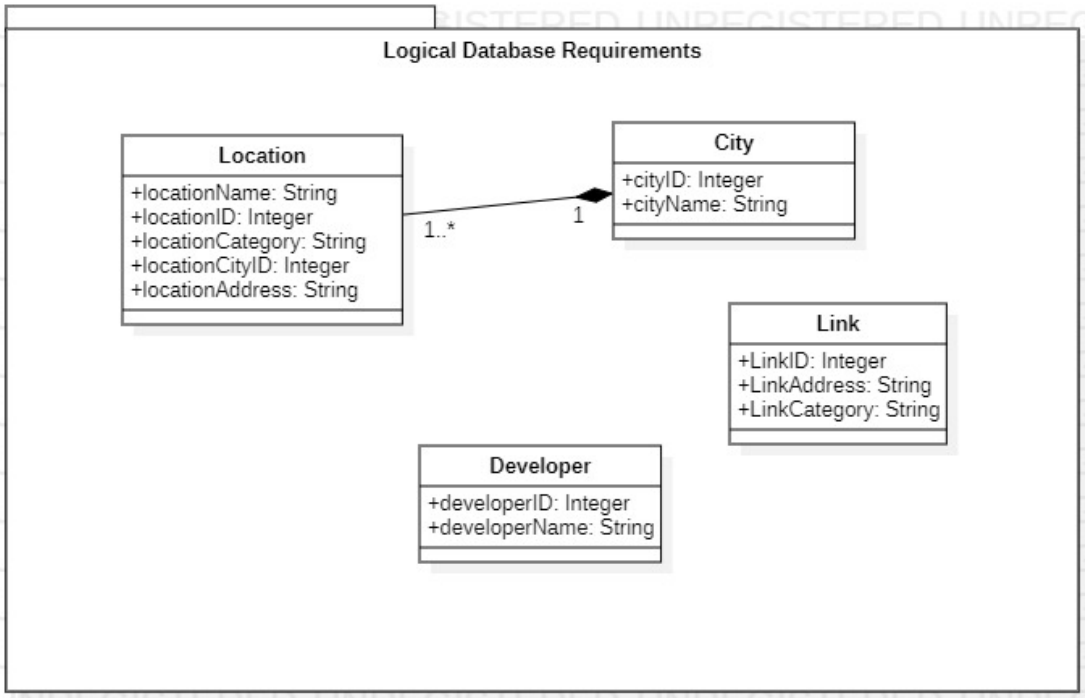

Figure 10: Db Requirements Class Diagram

- Locations are bounded to cities and cannot exist without a city / cityID.
- A city can have many locations but a location belongs to only one city.
- Other entities Link and Developer exist independently.
- Every entity has a unique id to separate them inside their class.
- When a city is deleted from the db, locations related to that city will also be removed.

- Location entity cannot exist without an address, an id and a city id.
- City cannot exist without both name and an id.
- Developer entity cannot exist without an id.
- Links cannot exist without address and an id.

4.3.3 Operations on Data

checkDeveloperCredentials	Read: Developer Create:- Update:- Delete:-
updateDeveloperName	Read:- Create:- Update:Developer Delete:-
createDeveloper	Read:- Create:Developer Update:- Delete:-
deleteDeveloper	Read:- Create:- Update:- Delete:Developer
createCity	Read:- Create:City Update:- Delete:-

deleteCity	Read:- Create:- Update:- Delete:City, Location
createLocation	Read:- Create:Location Update:- Delete:-
deleteLocation	Read:- Create:- Update:City Delete:Location
getCityInfo	Read:City Create:- Update:- Delete:-
getLocationInfo	Read:Location Create:- Update:- Delete:-
setLocationInfo	Read:- Create:- Update:Location Delete:-
setCity	Read:- Create:- Update:Location,City Delete:-

updateLocationInfo	Read:- Create:- Update:Location Delete:-
createLink	Read:- Create:Link Update:- Delete:-
deleteLink	Read:- Create:- Update:- Delete:Link
setLink	Read:- Create:- Update:Link Delete:-

Table 3: CRUD Operations

4.4 Deployment View

4.4.1 Stakeholders' uses of this view

This part of the system is particularly useful for the developers, which are one of our stakeholders, since it describes the environment into which the system will be deployed, as well as the dependencies. It gives a better idea regarding runtime platforms, software and network requirements, which are significant aspects after the software is built and validated.

4.4.2 Deployment Diagram

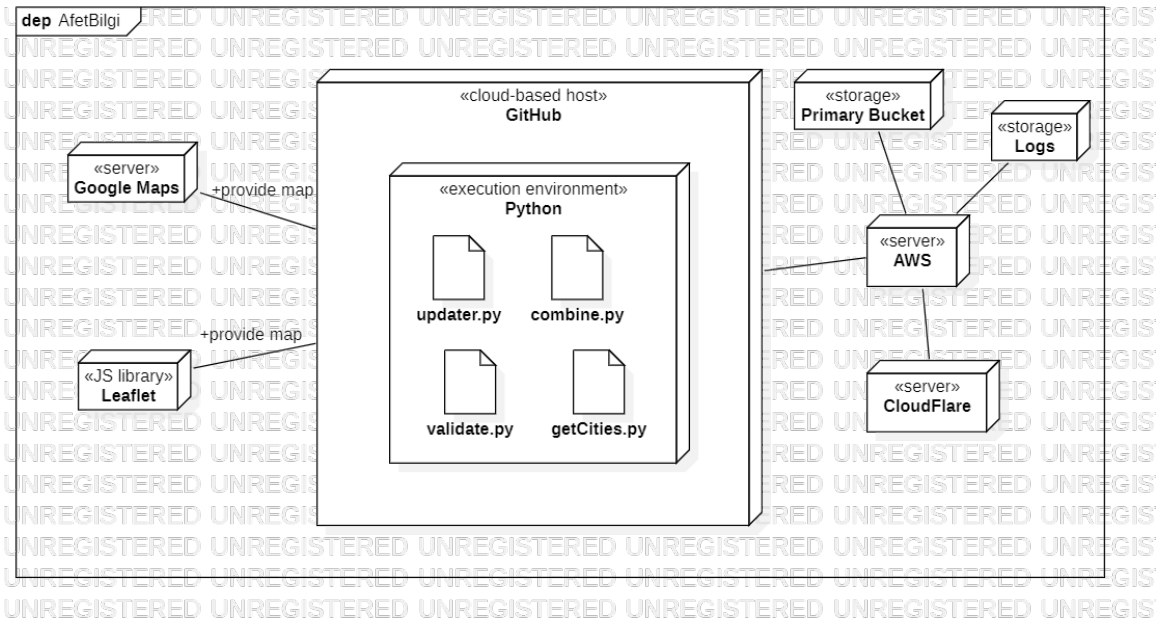

Figure 11: Deployment Diagram

- Inside the cloud-based host, the system includes some Python, execution environment, files, such as updater.py, combine.py, validate.py, getCities.py and so on.
- The System has servers , such as AWS, Cloudflare, and Google Maps in the deployment diagram.
- Python files represent their own particular behaviors that can easily be understood by their names.
- Primary Bucket and Logs are storages for Amazon Web Service, which is an important component of the system.

4.5 Design Rationale

4.5.1 Context View

The rationale behind the context view is to keep the system simple, reliable, and fast. Data is gathered from data collectors and validators, then this data is projected to the website by developers. The main purpose of the context view is to describe the relationships and interactions between the system and its environment. Activity diagrams were also helpful in clarifying different scenarios, which might have been difficult to understand only using the class diagram. So, the usage of context, class and activity diagrams allowed a broader and more detailed context view of the system.

4.5.2 Functional View

The rationale behind this view was the usage of component and class diagram (internal interfaces) in the functional view. Functional view in general shows the runtime functional elements of the system and their responsibilities. In this sense, the elements were mainly shown in the component diagram.the modularity and the simplicity of the functionality are the main concerns in this view.

4.5.3 Deployment View

The rationale behind the deployment view is keeping the overall software and hardware structure simple and modular. For that purpose, a deployment diagram was used as the main tool. As mentioned, the environments were portrayed more clearly in this way. The deployment view is the main concern of the developers, especially in our context, since the system includes all the necessary files related to different system components.

4.5.4 Information View

The rationale behind the information view is to provide users with fast and reliable information. The database is human-made and the data are verified by data collectors. The data collected are filled into Excel sheets and developers use the data accordingly. Simply they put the information from the data sheets into the pages of afetbilgi.com. The most important part of the information system is to be "fast" because natural disasters are emergency situations and the goal of the website is to help victims of natural disasters.

5 Architectural Views for Suggestions to Improve the Existing System

5.1 Context View

5.1.1 Stakeholders' uses of this view

There are two main stakeholders of the system: the users, and developers. Get Location feature is added by developers to the system which can be seen in the context diagram.

5.1.2 Context Diagram

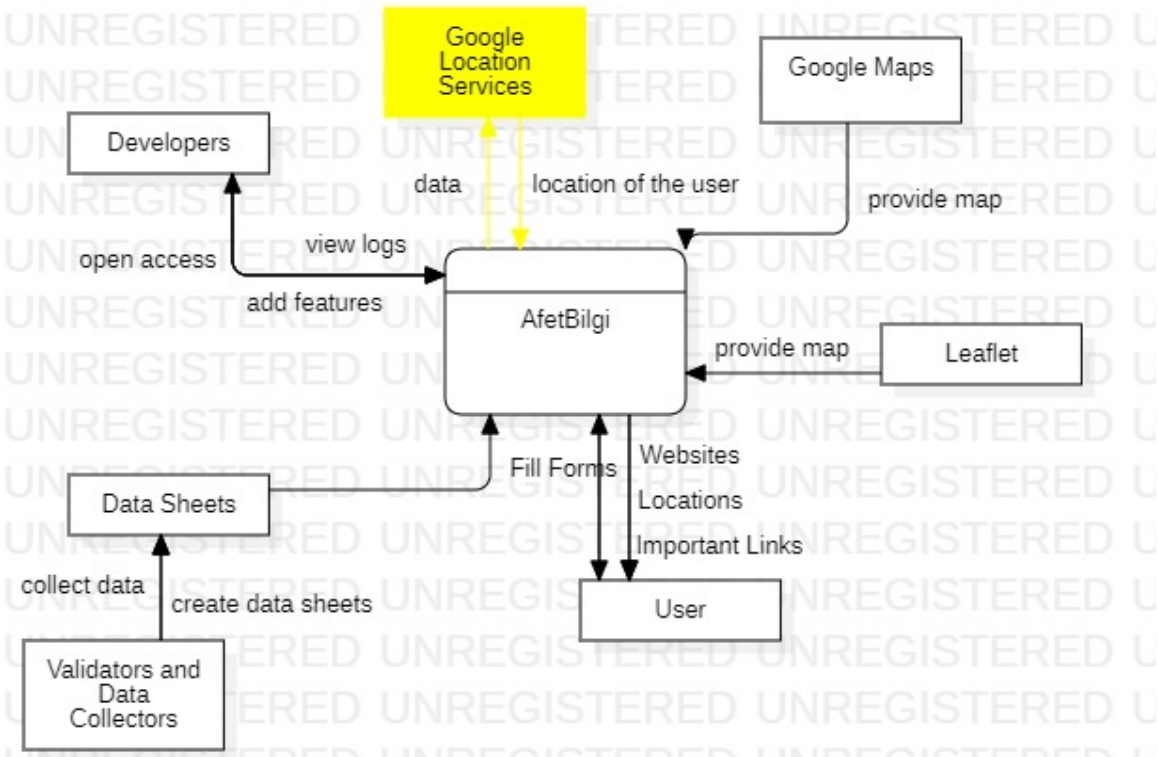

Figure 12: The new Context Diagram

- Developers view logs and can add features as the validators collect data after they accept the data as valid and reliable.
- Users can be provided with important links, sources, websites, locations, and related stuff. Also, they can fill out the forms on the website about a particular topic.
- Google Maps provide maps when users click to see some locations after some steps on the website.

- If the user clicks to see the map in the main interface of the system, Leaflet provides a map.
- Validators and data collectors create and provide data sheets to the developers after collecting and validating gathered data.

5.1.3 External Interfaces

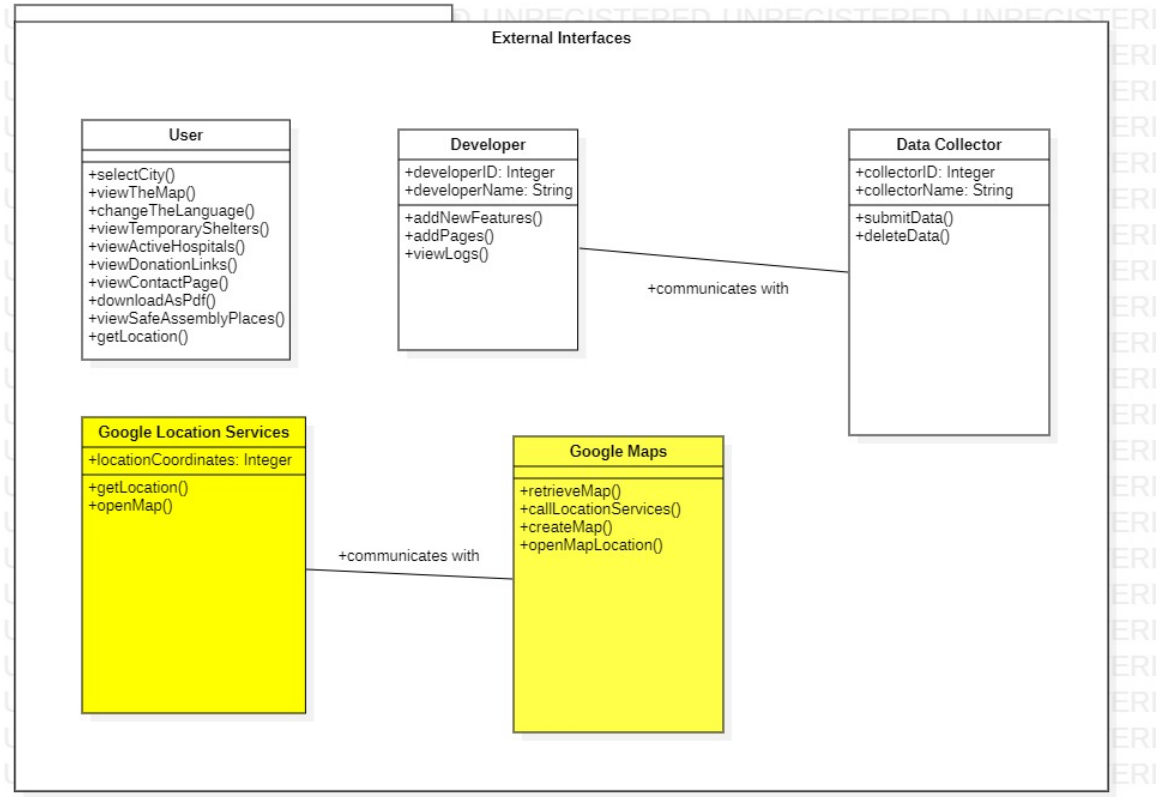

Figure 13: External Interfaces

Clear explanations of the diagram is provided in particular part of the Section 4.

5.1.4 Interaction scenarios

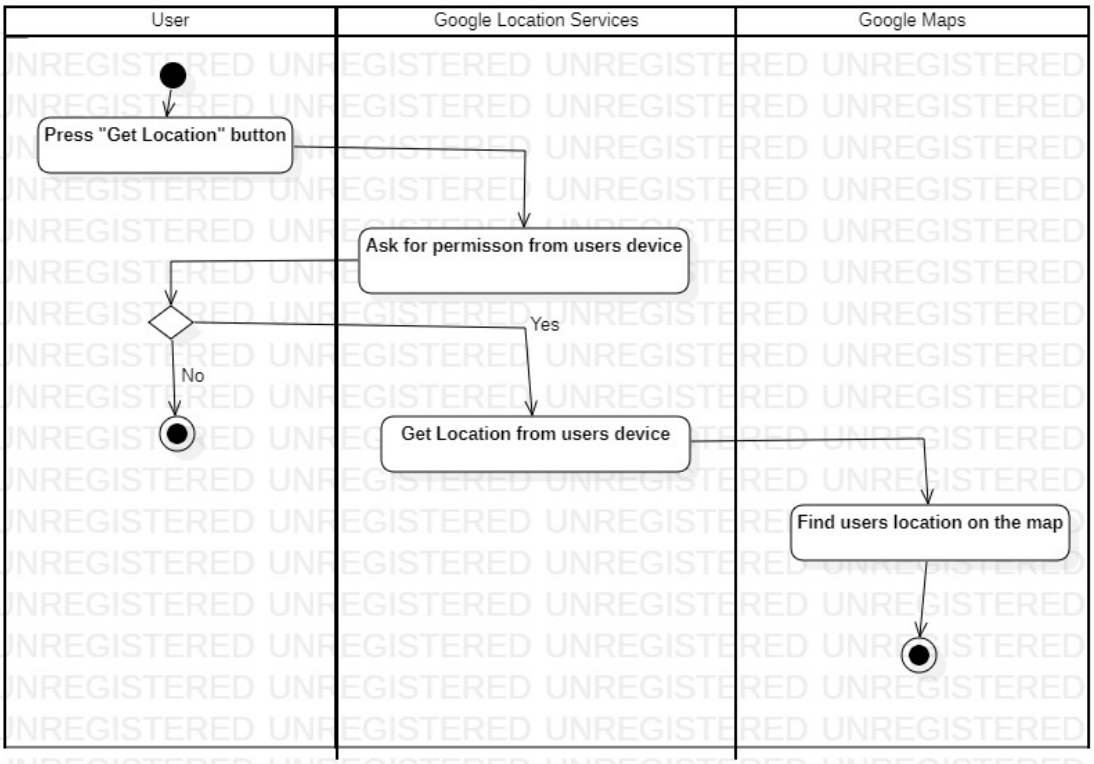

Figure 14: Activity Diagram of getLocation feature

5.2 Functional View

5.2.1 Stakeholders' uses of this view

In this viewpoint, different components of the system, internal interfaces within the system, their responsibilities and interactions will be described. Functional view is very significant to the stakeholders, since it provides an idea of the functionalities of the system and forms the shape of other viewpoints.

5.2.2 Component Diagram

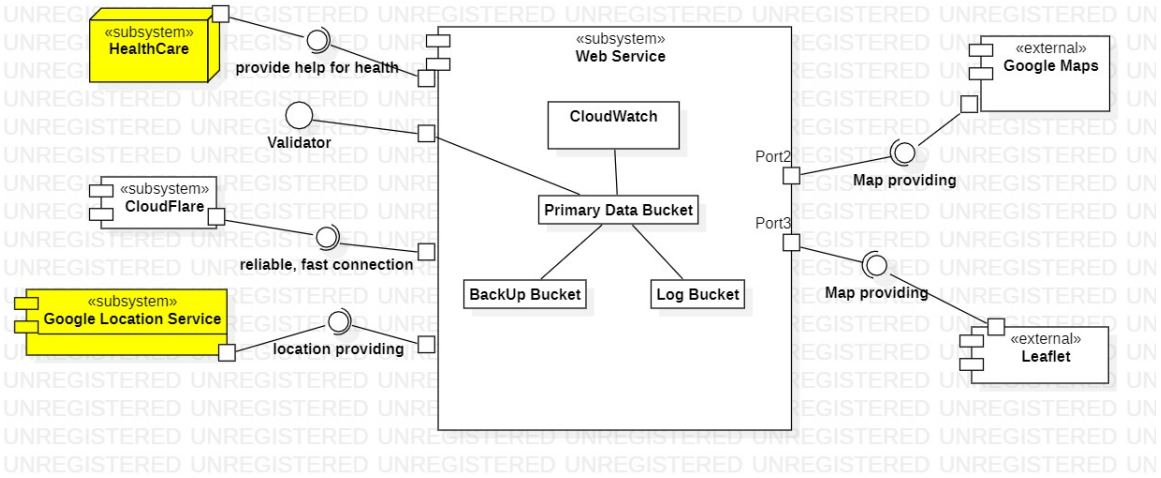

Figure 15: New Component Diagram

- Our system contains several subsystems, such as Web Services and CloudFlare, which involve the main work of the system.
- There are some external interfaces, such as Google Map, Leaflet, Validators, and Data Collectors.
- Web Service as a subsystem includes parts, including CloudWatch, Primary Data Bucket, BackUp Bucket, and Log Bucket, which are all inter-related.
- Web Service correlate with Cloudflare to provide secure, reliable, and fast connections to users.
- Validators correlates with subsystem Web Service and their role is to provide verified reliable data to developers and help their country. They get the verified data from organizations, twitter and other social media platforms.

- Google Maps and Leaflet as external interfaces provide maps to users in seperate parts of the system.
- The new feature "Get Location" is provided by Google Location Services which can retrieve users' locations from their devices. This feature can filter useless information that user does not have to see and users can use this feature to call for help if they are in trouble from the earthquake.
- Healthcare system can provide official and reliable health supplements to users in hard situations.

5.2.3 Internal Interfaces

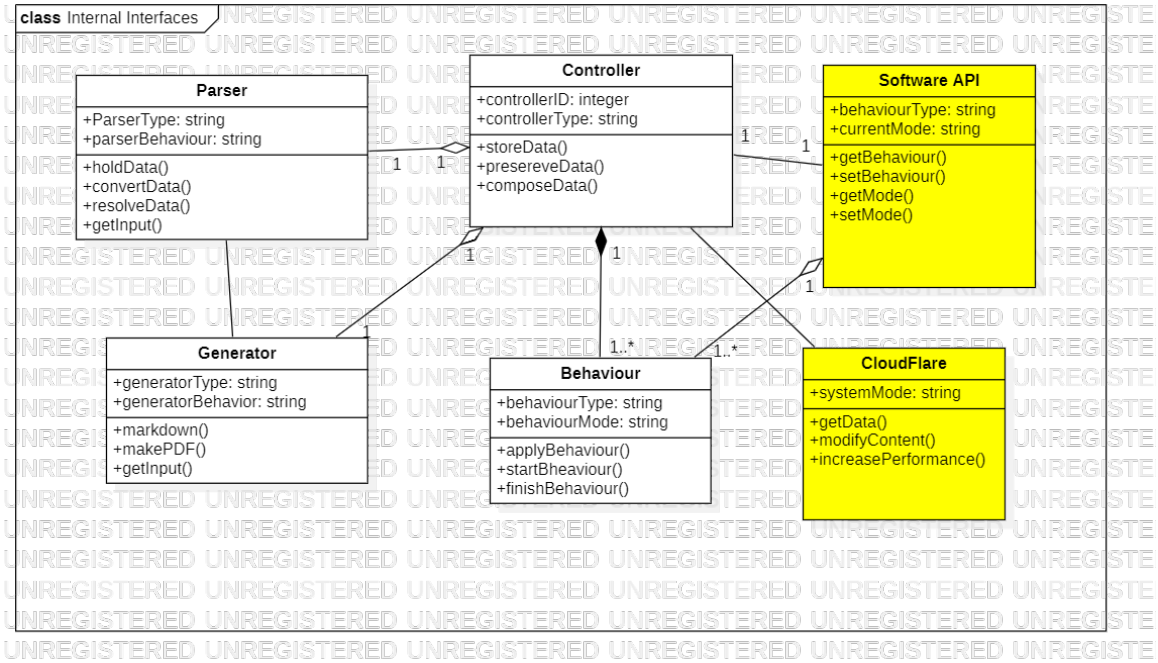

Figure 16: New Internal Interfaces

- Parser is a program that takes input data (usually source code) and converts it into a structured format. The purpose of parsing in Python is to transform code into a machine-readable format, allowing for further processing and analysis.

- Generator is a program that take input data and convert the data to particular form in order to make PDF file format.

- Behavior is characterized as the behaviour of the user while using the system.

- Controller will be responsible for composing the corresponding parts of the system. It will be responsible for storing the needed data in the system and preserving the integrity of the system.

- Software API and Cloudflare are new internal interfaces. Software API is contract of service between two systems. This contract means how the two communicate with each other using requests and responses. Cloudflare acts as an intermediary between a client and a server, and it allows the system to modify content for better performance.

5.2.4 Interaction Patterns

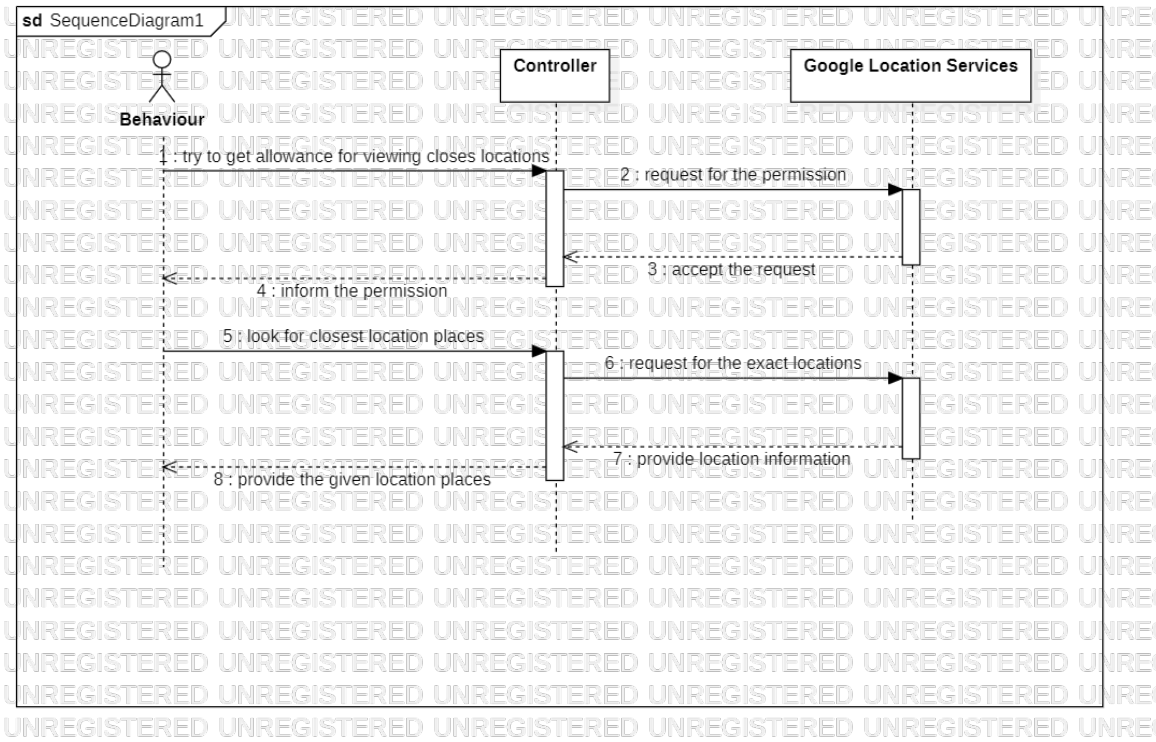

Figure 17: New Sequence Diagram for "Closest Locations"

5.3 Information View

5.3.1 Stakeholders' uses of this view

There are two main stakeholders of the system: Users and developers. Users use this view to understand how the data provided to them is related to each other and how it's formed. Users can understand what is stored in the system and its limitations. Developers use this view to see the relations of the db and its limitations. Also, they use this view to plan how to extend the db and the system without causing anomalies on the db.

5.3.2 Database Class Diagram

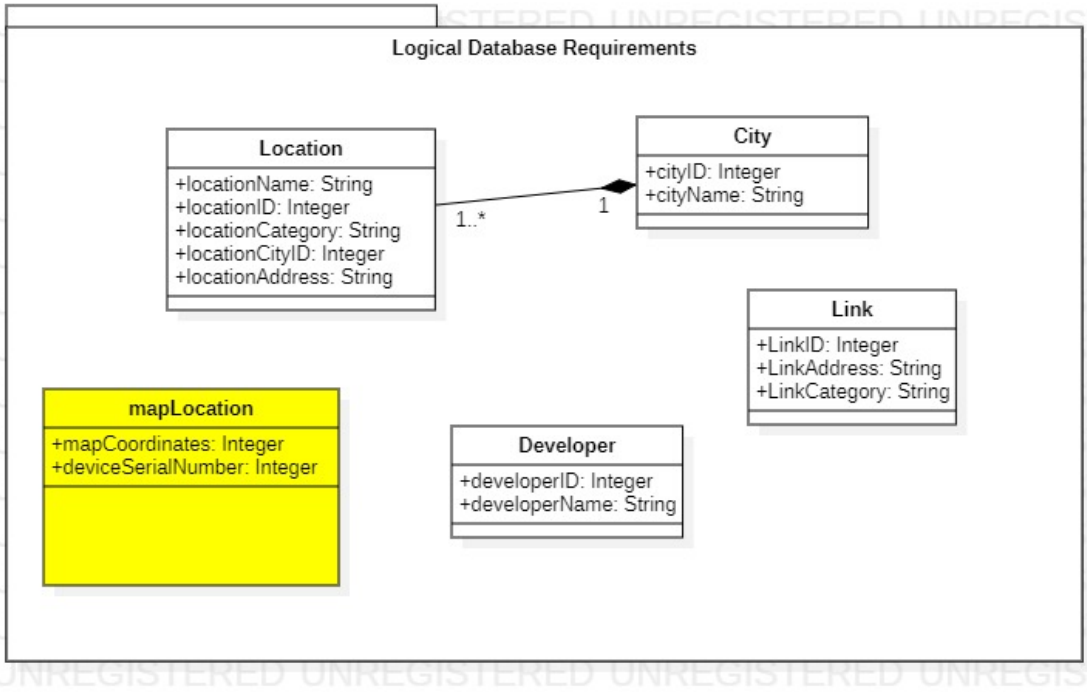

Figure 18: Updated Database Class Diagram

- mapCoordinates are generated from Google Location Services.
- Google Location Services also save the device serial number to identify the owner of the mobile phone.
- After mapCoordinates are retrieved, they are sent to the rescue organizations.
- mapLocation entities are independent and can exist only with coordinates. DeviceSerialNumber is not necessary.

5.3.3 Operations on Data

Here is table for additional operations on data. Current operations can be found in Section 4.

checkMapLocation	Read: mapLocation Create:- Update:- Delete:-
getMapLocation	Read:- Create:mapLocation Update:- Delete:-
getDeviceSerialNumber	Read:mapLocation Create:- Update:- Delete:-
deleteMapLocation	Read:- Create:- Update:- Delete:mapLocation

Table 4: CRUD Operations

5.4 Deployment View

5.4.1 Stakeholders' uses of this view

This part of the system is particularly useful for the developers, which are one of our stakeholders, since it describes the environment into which the system will be deployed, as well as the dependencies. It gives a better idea regarding runtime platforms, software and network

requirements, which are significant aspects after the software is built and validated.

5.4.2 Deployment Diagram

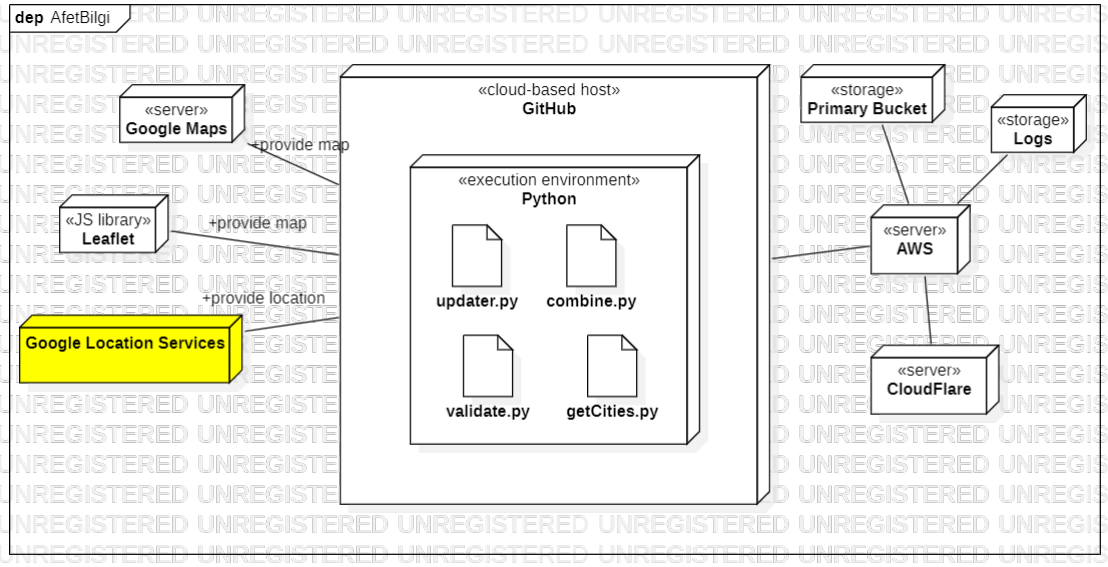

Figure 19: New Deployment Diagram

- Inside the cloud-based host, the system includes some Python, execution environment, files, such as updater.py, combine.py, validate.py, getCities.py and so on.
- The System has servers , such as AWS, Cloudflare, and Google Maps in the deployment diagram.
- Python files represent their own particular behaviors that can easily be understood by their names.

- Primary Bucket and Logs are storages for Amazon Web Service, which is an important component of the system.
- Google Location Services is the new subsystem which provide exact location of the user and the closest useful places very easily.

5.5 Design Rationale

5.5.1 Context View

The rationale behind the context view is to keep the system simple, reliable, and fast. Data is gathered from data collectors and validators, then this data is projected to the website by developers. The main purpose of the context view is to describe the relationships and interactions between the system and its environment. Activity diagrams were also helpful in clarifying different scenarios, which might have been difficult to understand only using the class diagram. So, the usage of context, class and activity diagrams allowed a broader and more detailed context view of the system.

5.5.2 Functional View

The rationale behind this view was the usage of component and class diagram (internal interfaces) in the functional view. Functional view in general shows the runtime functional elements of the system and their responsibilities. In this sense, the elements were mainly shown in the component diagram.the modularity and the simplicity of the functionality are the main concerns in this view.

5.5.3 Deployment View

The rationale behind the deployment view is keeping the overall software and hardware structure simple and modular. For that purpose, a

deployment diagram was used as the main tool. As mentioned, the environments were portrayed more clearly in this way. The deployment view is the main concern of the developers, especially in our context, since the system includes all the necessary files related to different system components.

5.5.4 Information View

The rationale behind the information view is to provide users with fast and reliable information. The database is human-made and the data are verified by data collectors. The data collected are filled into Excel sheets and developers use the data accordingly. Simply they put the information from the data sheets into the pages of afetbilgi.com. The most important part of the information system is to be "fast" because natural disasters are emergency situations and the goal of the website is to help victims of natural disasters.